

UNITED STATES PATENT AND TRADEMARK OFFICE (USPTO)

PROVISIONAL PATENT APPLICATION — SOURCE CODE APPENDIX

TITLE OF THE INVENTION:

ZERO-TRUST DATA SANITIZATION (ZTDS) ARCHITECTURE FOR EPHEMERAL IN-MEMORY PII REDACTION AND CRYPTOGRAPHIC ENCLAVE ATTESTATION IN ARTIFICIAL INTELLIGENCE WORKFLOWS

INVENTOR:

Ilya Sibiryakov (Tel Aviv, Israel)

APPENDIX OVERVIEW

This appendix forms an integral part of the patent specification and provides reproducible source code embodiments demonstrating reduction to practice of:

1. **Module 1:** Endpoint Volatile Sanitization & Rehydration Engine (`engine.py`)
 2. **Module 2:** Attestation Verifier & Nonce Lifecycle Controller (`verifier.py`)
 3. **Module 3:** Hardware Enclave Service & virtio-vsock Handler (`enclave_app.py`)
 4. **Module 4:** Zero-Server Cryptographic Session Handoff Engine (`crypto_handoff.js`)
-

MODULE 1: Endpoint Volatile Sanitization & Rehydration Engine (`engine.py`)

```
"""
ZTDS Client Volatile Sanitization Runtime (S_RAM) - Python Reference Engine
Executes in-memory DFA tokenization and reverse rehydration with zero disk
writes.
"""

import re
from typing import Dict, List, Tuple, Any

REGEX_PATTERNS = {
    'EMAIL': re.compile(r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'),
    'PHONE': re.compile(r'\b(?:\+?1[-.\s]?)?\([2-9]\d{2}\)[-.\s]? \d{3}[-.\s]? \d{4}\b'),
```

```

'SSN': re.compile(r'\b\d{3}-\d{2}-\d{4}\b'),
'EIN': re.compile(r'\b\d{2}-\d{7}\b'),
'CREDIT_CARD': re.compile(r'\b(?:\d{4}[-\s]){3}\d{4}\b'),
'IBAN': re.compile(r'\b[A-Z]{2}\d{2}[A-Z0-9]{4}\d{7}(?:[A-Z0-9]?){0,16}\b'),
'IPV4': re.compile(r'\b(?:?:25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.){3}(?:25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\b'),
'API_KEY': re.compile(r'\b(?:sk-[a-zA-Z0-9]{20,48}|AKIA[0-9A-Z]{16}|ghp_[a-zA-Z0-9]{36})\b'),
'MRN': re.compile(r'\bMRN[-\s]?d{6,10}\b', re.IGNORECASE),
'NAME': re.compile(r'\b(?:Dr\.|Mr\.|Ms\.|Mrs\.)\s+[A-Z][a-z]+(?:\s+[A-Z][a-z]+)?\b')
}

class ZTDSEngine:
    def __init__(self):
        self.session_map: Dict[str, str] = {}      # [TOKEN] -> original
        plaintext
        self.reverse_map: Dict[str, str] = {}      # original plaintext ->
        [TOKEN]
        self.entity_counters: Dict[str, int] = {}

    def reset_session(self):
        self.session_map.clear()
        self.reverse_map.clear()
        self.entity_counters.clear()

    def tokenize(self, text: str) -> Dict[str, Any]:
        if not text or not isinstance(text, str):
            return {'sanitized_text': '', 'token_count': 0, 'session_tokens':
            []}

        matches = []
        for entity_type, pattern in REGEX_PATTERNS.items():
            for m in pattern.finditer(text):
                matches.append({
                    'start': m.start(),
                    'end': m.end(),
                    'value': m.group(0),
                    'type': entity_type
                })

        # Sort descending by offset to prevent index drift
        matches.sort(key=lambda x: x['start'], reverse=True)

```

```

# Filter out overlapping intervals
filtered_matches = []
last_start = float('inf')
for match in matches:
    if match['end'] <= last_start:
        filtered_matches.append(match)
        last_start = match['start']

sanitized = text
for match in filtered_matches:
    token = self.reverse_map.get(match['value'])
    if not token:
        count = self.entity_counters.get(match['type'], 0) + 1
        self.entity_counters[match['type']] = count
        token = f"[{match['type']}]_{count}"
        self.session_map[token] = match['value']
        self.reverse_map[match['value']] = token

    sanitized = sanitized[:match['start']] + token +
sanitized[match['end']:]

return {
    'sanitized_text': sanitized,
    'token_count': len(filtered_matches),
    'session_tokens': list(self.session_map.keys())
}

def rehydrate(self, tokenized_response: str) -> str:
    if not tokenized_response or not isinstance(tokenized_response, str):
        return ''

    rehydrated = tokenized_response
    for token, original_val in self.session_map.items():
        rehydrated = rehydrated.replace(token, original_val)

    return rehydrated

```

MODULE 2: Attestation Verifier & Nonce Lifecycle Controller (`verifier.py`)

```

"""
ZTDS Attestation Verifier (AV) & Nonce Engine
Validates COSE_Sign1 documents, checks PCR measurements against approved
policy,

```

and verifies freshness of the 128-bit challenge nonce under a fail-closed policy.

```
"""
```

```
import time
import secrets
import json
```

```
APPROVED_PCR0 =
"a1b2c3d4e5f60718293a4b5c6d7e8f90123456789abcdef0123456789abcdef0"
MAX_NONCE_AGE_SEC = 300 # 5 minutes
```

```
class AttestationVerifier:
    def __init__(self):
        self.active_challenges = {}

    def generate_challenge(self, site_id: str) -> str:
        nonce = secrets.token_bytes(16)
        nonce_hex = nonce.hex()
        self.active_challenges[nonce_hex] = {
            "site_id": site_id,
            "created_at": time.time()
        }
        return nonce_hex

    def verify_evidence(self, attestation_doc: dict) -> tuple:
        nonce_hex = attestation_doc.get("nonce_hex")
        if not nonce_hex or nonce_hex not in self.active_challenges:
            return False, "REJECT: Challenge nonce missing or unknown"

        challenge_data = self.active_challenges.pop(nonce_hex)
        age = time.time() - challenge_data["created_at"]
        if age > MAX_NONCE_AGE_SEC:
            return False, f"REJECT: Stale nonce (age: {age:.1f}s > {MAX_NONCE_AGE_SEC}s)"

        pcrs = attestation_doc.get("pcrs", {})
        pcr0 = pcrs.get("pcr0")
        if pcr0 != APPROVED_PCR0:
            return False, f"REJECT: PCR0 mismatch (got {pcr0}, expected {APPROVED_PCR0})"

        return True, "ACCEPT: Attestation evidence verified successfully"
```

MODULE 3: Hardware Enclave Service & virtio-vsock Handler (`enclave_app.py`)

```
"""
ZTDS AWS Nitro Enclave Server
Listens on AF_VSOCK for:
1. Attestation Evidence Request (Port 5000)
2. Sanitized Query Dispatch (Port 5005)
"""

import socket
import json
import os

VSOCK_PORT_ATTESTATION = 5000
VSOCK_PORT_QUERY = 5005

def handle_attestation_request(nonce_bytes):
    """
    Interacts with NSM to generate signed hardware attestation evidence.
    """
    mock_attestation_doc = {
        "module_id": "ztds-site1-nitro",
        "timestamp_ms": 1788415000,
        "pcrs": {
            "pcr0":
"a1b2c3d4e5f60718293a4b5c6d7e8f90123456789abcdef0123456789abcdef0",
            "pcr1":
"0000000000000000000000000000000000000000000000000000000000000000",
            "pcr2":
"fedcba9876543210fedcba9876543210fedcba9876543210fedcba9876543210"
        },
        "nonce_hex": nonce_bytes.hex() if isinstance(nonce_bytes, bytes) else
str(nonce_bytes),
        "public_key": "MOCK_ENCLAVE_PUBKEY_SECP384R1",
        "signature": "MOCK_NITRO_HYPERVISOR_SIG"
    }
    return mock_attestation_doc

def process_query(tokenized_query):
    """
    Executes private enclave workload over tokenized inputs.
    Preserves surrogate tokens intact without accessing external networks.
    """
```

```
    return f"ENCLAVE_RESPONSE: Successfully processed prompt:
{tokenized_query}. All private models executed inside hardware enclave."
```

MODULE 4: Zero-Server Cryptographic Session Handoff Engine (`crypto_handoff.js`)

```
/**
 * ZTDS Zero-Server Cryptographic Session Handoff Engine
 * Implements client-side Argon2id key derivation and XChaCha20-Poly1305 AEAD
 * packaging.
 */

const TEAM_VERSION = 4; // Version 4: Argon2id + XChaCha20-Poly1305

async function packSessionHandoff(sessionMap, passphrase, libsodium) {
  if (!passphrase || passphrase.length < 8) {
    throw new Error('Passphrase must be at least 8 characters.');
  }
  const sodium = libsodium;

  // 1. Generate 16-byte random salt and 24-byte extended nonce
  const salt = sodium.randombytes_buf(sodium.crypto_pwhash_SALTBYTES);
  const nonce =
sodium.randombytes_buf(sodium.crypto_aead_xchacha20poly1305_ietf_NPUBBYTES);

  // 2. Derive 256-bit symmetric key using memory-hard Argon2id (v0x13)
  const key = sodium.crypto_pwhash(
    sodium.crypto_aead_xchacha20poly1305_ietf_KEYBYTES,
    passphrase,
    salt,
    sodium.crypto_pwhash_OPSLIMIT_MODERATE,
    sodium.crypto_pwhash_MEMLIMIT_MODERATE,
    sodium.crypto_pwhash_ALG_ARGON2ID13
  );

  // 3. Serialize session mapping payload
  const payload = {
    v: TEAM_VERSION,
    sm: sessionMap,
    ts: Date.now()
  };
  const plaintext = new TextEncoder().encode(JSON.stringify(payload));

  // 4. Encrypt using XChaCha20-Poly1305 authenticated encryption
```

```

    const ciphertext = sodium.crypto_aead_xchacha20poly1305_ietf_encrypt(
      plaintext,
      null,
      null,
      nonce,
      key
    );

    // 5. Construct binary container: [Version(1B) | Salt(16B) | Nonce(24B) |
    Ciphertext + Tag]
    const container = new Uint8Array(1 + salt.length + nonce.length +
    ciphertext.length);
    container[0] = TEAM_VERSION;
    container.set(salt, 1);
    container.set(nonce, 1 + salt.length);
    container.set(ciphertext, 1 + salt.length + nonce.length);

    // 6. Cryptographically wipe key from volatile memory
    sodium.memzero(key);

    return container;
  }

  async function unpackSessionHandoff(containerBytes, passphrase, libsodium) {
    const version = containerBytes[0];
    if (version !== TEAM_VERSION) {
      throw new Error('Unsupported container version.');
```

```

        sodium.crypto_pwhash_MEMLIMIT_MODERATE,
        sodium.crypto_pwhash_ALG_ARGON2ID13
    );

    try {
        // Authenticated decryption under fail-closed verification
        const decryptedRaw =
sodium.crypto_aead_xchacha20poly1305_ietf_decrypt(
            null,
            ciphertext,
            null,
            nonce,
            key
        );
        const jsonStr = new TextDecoder().decode(decryptedRaw);
        return JSON.parse(jsonStr);
    } finally {
        // Invariant memory wipe
        sodium.memzero(key);
    }
}

```